

C++ OOP Revision: Complete Questions and Solutions

Prepared for: Sob Fred Date: May 1, 2026

Table of Contents

1. Q1: University Staff Management System
 - Abstract Base Class StaffMember
 - Derived Class Lecturer
 - Derived Class Administrator
 - Polymorphism in main()
 2. Q2: Smart Transport and Ride Management System
 - QUESTION 1: Classes and Objects
 - QUESTION 2: Encapsulation
 - QUESTION 3: Inheritance
 - QUESTION 4: Real-Life System Integration
 3. Additional OOP Questions
 - Q3: Library Management System
 - Q4: Bank Account System
 - Q5: Shape Hierarchy
 - Q6: Employee Management System
 - Q7: Animal Kingdom Simulation
 4. Summary of OOP Concepts
-
-

Q1: University Staff Management System

(a) Abstract Base Class StaffMember

An **abstract base class** with a pure virtual function computeMonthlyPay().

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class StaffMember {
```

```

private:
    string name;
    int id;
    double baseSalary;

public:
    StaffMember(string n, int i, double salary) : name(n), id(i)
    {
        if (salary > 0) baseSalary = salary;
        else baseSalary = 0;
    }

    // Getters
    string getName() const { return name; }
    int getId() const { return id; }
    double getBaseSalary() const { return baseSalary; }

    // Setter with validation
    void setBaseSalary(double salary) {
        if (salary > 0) baseSalary = salary;
    }

    // Pure virtual function
    virtual double computeMonthlyPay() const = 0;

    // Virtual destructor
    virtual ~StaffMember() {}
};

```

Key Points:

- **Encapsulation:** Private attributes with public getters/setters.
 - **Validation:** Ensures baseSalary is positive.
 - **Abstraction:** computeMonthlyPay() is pure virtual.
-

(b) Derived Class Lecturer

Implements computeMonthlyPay() as:
baseSalary + (hoursPerWeek × 5000 × 4).

```

class Lecturer : public StaffMember {
private:
    int hoursPerWeek;

```

```

public:
    Lecturer(string n, int i, double salary, int hours)
        : StaffMember(n, i, salary), hoursPerWeek(hours) {}

    double computeMonthlyPay() const override {
        return getBaseSalary() + (hoursPerWeek * 5000 * 4);
    }

    int getHoursPerWeek() const { return hoursPerWeek; }
};

```

Key Points:

- **Inheritance:** Extends StaffMember.
 - **Method Overriding:** Implements computeMonthlyPay().
-

(c) Derived Class Administrator

Implements computeMonthlyPay() as $\text{baseSalary} \times 1.2$.

```

class Administrator : public StaffMember {
private:
    string department;

public:
    Administrator(string n, int i, double salary, string dept)
        : StaffMember(n, i, salary), department(dept) {}

    double computeMonthlyPay() const override {
        return getBaseSalary() * 1.2;
    }

    string getDepartment() const { return department; }
};

```

Key Points:

- **Polymorphism:** Different implementation of computeMonthlyPay().
-

(d) Polymorphism in main()

Demonstrate polymorphism with a vector<StaffMember*>.

```

int main() {
    StaffMember* staff1 = new Lecturer("Dr. Smith", 101, 500000,
10);
    StaffMember* staff2 = new Administrator("Mr. Jones", 102,
400000, "Finance");

    vector<StaffMember*> staffList = {staff1, staff2};

    for (StaffMember* staff : staffList) {
        cout << "Name: " << staff->getName()
            << ", Monthly Pay: " << staff->computeMonthlyPay()
<< endl;
    }

    for (StaffMember* staff : staffList) {
        delete staff;
    }

    return 0;
}

```

Output:

Name: Dr. Smith, Monthly Pay: 700000
Name: Mr. Jones, Monthly Pay: 480000

Key Points:

- **Polymorphism:** Correct computeMonthlyPay() called for each object.
- **Memory Management:** delete used to free memory.

Q2: Smart Transport and Ride Management System

QUESTION 1: Classes and Objects

(a) Class Design for Vehicle

```

class Vehicle {
private:
    string vehicleID;
    string brand;
    float fuelLevel;
    float mileage;
}

```

```

public:
    Vehicle(string id, string b, float fuel, float km)
        : vehicleID(id), brand(b), fuelLevel(fuel), mileage(km)
    {}

    void displayVehicleInfo() const {
        cout << "Vehicle ID: " << vehicleID << endl;
        cout << "Brand: " << brand << endl;
        cout << "Fuel Level: " << fuelLevel << "L" << endl;
        cout << "Mileage: " << mileage << "km" << endl;
    }

    void updateFuel(float amount) {
        if (amount > 0) fuelLevel += amount;
        else cout << "Invalid fuel amount!" << endl;
    }

    string getVehicleID() const { return vehicleID; }
    string getBrand() const { return brand; }
    float getFuelLevel() const { return fuelLevel; }
    float getMileage() const { return mileage; }
};

```

Key Points:

- **Encapsulation:** Private attributes with public methods.
- **Validation:** updateFuel() checks for positive values.

(b) Object Implementation in main()

```

int main() {
    Vehicle car1("V001", "Toyota", 45.5, 15000.0);
    Vehicle car2("V002", "Honda", 30.0, 20000.0);
    Vehicle truck1("T001", "Ford", 100.0, 50000.0);

    car1.displayVehicleInfo();
    car1.updateFuel(10.0);
    car1.displayVehicleInfo();

    return 0;
}

```

Output:

Vehicle ID: V001
Brand: Toyota
Fuel Level: 45.5L
Mileage: 15000km

Vehicle ID: V001
Brand: Toyota
Fuel Level: 55.5L
Mileage: 15000km

QUESTION 2: Encapsulation

(a) Class Design for Driver

```
class Driver {
private:
    string driverID;
    string name;
    string licenseNumber;
    float rating;

public:
    Driver(string id, string n, string license, float r)
        : driverID(id), name(n), licenseNumber(license) {
        setRating(r);
    }

    string getDriverID() const { return driverID; }
    string getName() const { return name; }
    string getLicenseNumber() const { return licenseNumber; }
    float getRating() const { return rating; }

    void setRating(float r) {
        if (r >= 0 && r <= 5) rating = r;
        else cout << "Rating must be between 0 and 5!" << endl;
    }
};
```

Key Points:

- **Encapsulation:** rating is private; modified via setRating().
 - **Validation:** Ensures rating is between 0 and 5.
-

(b) Demonstration of Encapsulation

```
int main() {
    Driver driver1("D001", "Alice", "LIC123", 4.5);
    cout << "Driver: " << driver1.getName() << ", Rating: " <<
driver1.getRating() << endl;

    driver1.setRating(6.0);
    driver1.setRating(4.8);
    cout << "Updated Rating: " << driver1.getRating() << endl;

    return 0;
}
```

Output:

```
Driver: Alice, Rating: 4.5
Rating must be between 0 and 5!
Updated Rating: 4.8
```

QUESTION 3: Inheritance

(a) Base Class Vehicle

Already defined above.

(b) Derived Classes Car and Truck

```
class Car : public Vehicle {
private:
    int numberOfSeats;

public:
    Car(string id, string b, float fuel, float km, int seats)
        : Vehicle(id, b, fuel, km), numberOfSeats(seats) {}

    void displayVehicleInfo() const override {
        Vehicle::displayVehicleInfo();
        cout << "Number of Seats: " << numberOfSeats << endl;
    }

    int getNumberOfSeats() const { return numberOfSeats; }
};
```

```

class Truck : public Vehicle {
private:
    float loadCapacity;

public:
    Truck(string id, string b, float fuel, float km, float
capacity)
        : Vehicle(id, b, fuel, km), loadCapacity(capacity) {}

    void displayVehicleInfo() const override {
        Vehicle::displayVehicleInfo();
        cout << "Load Capacity: " << loadCapacity << " tons" <<
endl;
    }

    float getLoadCapacity() const { return loadCapacity; }
};

```

(c) Demonstration in main()

```

int main() {
    Car car("C001", "Toyota", 50.0, 10000.0, 5);
    Truck truck("T001", "Ford", 150.0, 50000.0, 10.0);

    Vehicle* vehicle1 = &car;
    Vehicle* vehicle2 = &truck;

    vehicle1->displayVehicleInfo();
    vehicle2->displayVehicleInfo();

    return 0;
}

```

Output:

```

Vehicle ID: C001
Brand: Toyota
Fuel Level: 50L
Mileage: 10000km
Number of Seats: 5

```

```

Vehicle ID: T001
Brand: Ford

```


Fuel Level: 150L
Mileage: 50000km
Load Capacity: 10 tons

QUESTION 4: Real-Life System Integration

(a) Ride Class

```
class Ride {
private:
    string rideID;
    Driver driver;
    Vehicle vehicle;
    float distance;
    float fare;

public:
    Ride(string id, Driver d, Vehicle v, float dist)
        : rideID(id), driver(d), vehicle(v), distance(dist) {
        fare = calculateFare();
    }

    float calculateFare() const {
        return distance * 1000;
    }

    void displayRideDetails() const {
        cout << "Ride ID: " << rideID << endl;
        cout << "Driver: " << driver.getName() << " (Rating: " <<
driver.getRating() << ")" << endl;
        cout << "Vehicle: " << vehicle.getBrand() << " (" <<
vehicle.getVehicleID() << ")" << endl;
        cout << "Distance: " << distance << "km" << endl;
        cout << "Fare: " << fare << " FCFA" << endl;
    }
};
```

(b) System Simulation in main()

```
int main() {
    Driver driver1("D001", "Alice", "LIC123", 4.5);
    Driver driver2("D002", "Bob", "LIC456", 4.8);
```

```
Car car("C001", "Toyota", 50.0, 10000.0, 5);
Truck truck("T001", "Ford", 150.0, 50000.0, 10.0);

Ride ride1("R001", driver1, car, 25.0);
Ride ride2("R002", driver2, truck, 100.0);

ride1.displayRideDetails();
cout << "---" << endl;
ride2.displayRideDetails();

return 0;
}
```

Output:

```
Ride ID: R001
Driver: Alice (Rating: 4.5)
Vehicle: Toyota (C001)
Distance: 25km
Fare: 25000 FCFA
---
Ride ID: R002
Driver: Bob (Rating: 4.8)
Vehicle: Ford (T001)
Distance: 100km
Fare: 100000 FCFA
```

Additional OOP Questions

Q3: Library Management System

(a) Base Class: Book

```
class Book {
protected:
    string title;
    string author;
    string ISBN;
    double price;
```

```

public:
    Book(string t, string a, string isbn, double p)
        : title(t), author(a), ISBN(isbn), price(p) {}

    virtual void displayDetails() const {
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "ISBN: " << ISBN << endl;
        cout << "Price: " << price << " FCFA" << endl;
    }

    virtual double applyDiscount(double percentage) {
        if (percentage > 0 && percentage <= 100) {
            price *= (1 - percentage / 100);
        }
        return price;
    }

    double getPrice() const { return price; }
};

```

(b) Derived Classes: FictionBook and TextBook

```

class FictionBook : public Book {
private:
    string genre;

public:
    FictionBook(string t, string a, string isbn, double p, string
g)
        : Book(t, a, isbn, p), genre(g) {}

    void displayDetails() const override {
        Book::displayDetails();
        cout << "Genre: " << genre << endl;
    }

    double applyDiscount(double percentage = 10) override {
        return Book::applyDiscount(percentage);
    }
};

class TextBook : public Book {

```

```

private:
    int edition;
    string subject;

public:
    TextBook(string t, string a, string isbn, double p, int e,
string s)
        : Book(t, a, isbn, p), edition(e), subject(s) {}

    void displayDetails() const override {
        Book::displayDetails();
        cout << "Edition: " << edition << endl;
        cout << "Subject: " << subject << endl;
    }

    double applyDiscount(double percentage = 5) override {
        return Book::applyDiscount(percentage);
    }
};

```

(c) Polymorphism in main()

```

int main() {
    Book* book1 = new FictionBook("The Hobbit", "J.R.R. Tolkien",
"ISBN123", 5000, "Fantasy");
    Book* book2 = new TextBook("C++ Programming", "Bjarne
Stroustrup", "ISBN456", 8000, 3, "Computer Science");

    vector<Book*> books = {book1, book2};

    for (Book* book : books) {
        book->applyDiscount();
        book->displayDetails();
        cout << "---" << endl;
    }

    for (Book* book : books) {
        delete book;
    }

    return 0;
}

```

Output:

Title: The Hobbit
Author: J.R.R. Tolkien
ISBN: ISBN123
Price: 4500 FCFA
Genre: Fantasy

Title: C++ Programming
Author: Bjarne Stroustrup
ISBN: ISBN456
Price: 7600 FCFA
Edition: 3
Subject: Computer Science

Q4: Bank Account System

(a) Abstract Base Class: Account

```
class Account {
protected:
    string accountNumber;
    string accountHolder;
    double balance;

public:
    Account(string accNum, string accHolder, double bal)
        : accountNumber(accNum), accountHolder(accHolder),
        balance(bal) {}

    virtual void deposit(double amount) = 0;
    virtual void withdraw(double amount) = 0;
    virtual void display() const = 0;

    double getBalance() const { return balance; }
    virtual ~Account() {}
};
```

(b) Derived Classes: SavingsAccount and CurrentAccount

```
class SavingsAccount : public Account {
private:
    double interestRate;
```

```

public:
    SavingsAccount(string accNum, string accHolder, double bal,
double rate)
        : Account(accNum, accHolder, bal), interestRate(rate) {}

    void deposit(double amount) override {
        if (amount > 0) {
            balance += amount;
            cout << "Deposited: " << amount << " FCFA. New
balance: " << balance << " FCFA" << endl;
        } else {
            cout << "Invalid deposit amount!" << endl;
        }
    }

    void withdraw(double amount) override {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            cout << "Withdrew: " << amount << " FCFA. New
balance: " << balance << " FCFA" << endl;
        } else {
            cout << "Insufficient funds or invalid amount!" <<
endl;
        }
    }

    void display() const override {
        cout << "Savings Account" << endl;
        cout << "Account Number: " << accountNumber << endl;
        cout << "Account Holder: " << accountHolder << endl;
        cout << "Balance: " << balance << " FCFA" << endl;
        cout << "Interest Rate: " << interestRate * 100 << "%" <<
endl;
    }

    void applyInterest() {
        balance += balance * interestRate;
        cout << "Interest applied. New balance: " << balance << "
FCFA" << endl;
    }
};

class CurrentAccount : public Account {

```

```

private:
    double overdraftLimit;

public:
    CurrentAccount(string accNum, string accHolder, double bal,
double limit)
        : Account(accNum, accHolder, bal), overdraftLimit(limit)
    {}

    void deposit(double amount) override {
        if (amount > 0) {
            balance += amount;
            cout << "Deposited: " << amount << " FCFA. New
balance: " << balance << " FCFA" << endl;
        } else {
            cout << "Invalid deposit amount!" << endl;
        }
    }

    void withdraw(double amount) override {
        if (amount > 0 && (balance - amount) >= -overdraftLimit)
{
            balance -= amount;
            cout << "Withdrew: " << amount << " FCFA. New
balance: " << balance << " FCFA" << endl;
        } else {
            cout << "Exceeds overdraft limit or invalid amount!"
<< endl;
        }
    }

    void display() const override {
        cout << "Current Account" << endl;
        cout << "Account Number: " << accountNumber << endl;
        cout << "Account Holder: " << accountHolder << endl;
        cout << "Balance: " << balance << " FCFA" << endl;
        cout << "Overdraft Limit: " << overdraftLimit << " FCFA"
<< endl;
    }
};

```

(c) Demonstration in main()

```
int main() {
    Account* acc1 = new SavingsAccount("SA001", "Alice", 10000,
0.05);
    Account* acc2 = new CurrentAccount("CA001", "Bob", 5000,
2000);

    vector<Account*> accounts = {acc1, acc2};

    for (Account* acc : accounts) {
        acc->deposit(2000);
        acc->withdraw(1000);
        acc->display();
        cout << "---" << endl;
    }

    SavingsAccount* savingsAcc =
dynamic_cast<SavingsAccount*>(acc1);
    if (savingsAcc) {
        savingsAcc->applyInterest();
    }

    for (Account* acc : accounts) {
        delete acc;
    }

    return 0;
}
```

Output:

```
Deposited: 2000 FCFA. New balance: 12000 FCFA
Withdrew: 1000 FCFA. New balance: 11000 FCFA
Savings Account
Account Number: SA001
Account Holder: Alice
Balance: 11000 FCFA
Interest Rate: 5%
---
Deposited: 2000 FCFA. New balance: 7000 FCFA
Withdrew: 1000 FCFA. New balance: 6000 FCFA
Current Account
Account Number: CA001
Account Holder: Bob
```


Balance: 6000 FCFA

Overdraft Limit: 2000 FCFA

Interest applied. New balance: 11550 FCFA

Q5: Shape Hierarchy

(a) Base Class: Shape

```
#include <cmath>
class Shape {
public:
    virtual double area() const = 0;
    virtual double perimeter() const = 0;
    virtual ~Shape() {}
};
```

(b) Derived Classes: Circle, Rectangle, and Triangle

```
class Circle : public Shape {
private:
    double radius;

public:
    Circle(double r) : radius(r) {}

    double area() const override {
        return M_PI * radius * radius;
    }

    double perimeter() const override {
        return 2 * M_PI * radius;
    }
};

class Rectangle : public Shape {
private:
    double length;
    double width;

public:
    Rectangle(double l, double w) : length(l), width(w) {}
```

```

    double area() const override {
        return length * width;
    }

    double perimeter() const override {
        return 2 * (length + width);
    }
};

class Triangle : public Shape {
private:
    double side1, side2, side3;

public:
    Triangle(double s1, double s2, double s3) : side1(s1),
    side2(s2), side3(s3) {}

    double area() const override {
        double s = (side1 + side2 + side3) / 2;
        return sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    double perimeter() const override {
        return side1 + side2 + side3;
    }
};

```

(c) Polymorphism and Virtual Functions

```

int main() {
    Shape* shape1 = new Circle(5.0);
    Shape* shape2 = new Rectangle(4.0, 6.0);
    Shape* shape3 = new Triangle(3.0, 4.0, 5.0);

    vector<Shape*> shapes = {shape1, shape2, shape3};

    for (Shape* shape : shapes) {
        cout << "Area: " << shape->area() << endl;
        cout << "Perimeter: " << shape->perimeter() << endl;
        cout << "---" << endl;
    }
}

```

```

        for (Shape* shape : shapes) {
            delete shape;
        }

        return 0;
    }

```

Output:

```

Area: 78.5398
Perimeter: 31.4159
---
Area: 24
Perimeter: 20
---
Area: 6
Perimeter: 12
---

```

Q6: Employee Management System

(a) Base Class: Employee

```

class Employee {
protected:
    string name;
    int id;
    double salary;

public:
    Employee(string n, int i, double s) : name(n), id(i),
    salary(s) {}

    virtual double calculateBonus() const = 0;

    virtual void displayDetails() const {
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Salary: " << salary << " FCFA" << endl;
    }

    virtual ~Employee() {}
};

```

(b) Derived Classes: FullTimeEmployee and PartTimeEmployee

```
class FullTimeEmployee : public Employee {
private:
    int yearsOfService;

public:
    FullTimeEmployee(string n, int i, double s, int years)
        : Employee(n, i, s), yearsOfService(years) {}

    double calculateBonus() const override {
        return salary * 0.1 * yearsOfService;
    }

    void displayDetails() const override {
        Employee::displayDetails();
        cout << "Years of Service: " << yearsOfService << endl;
        cout << "Bonus: " << calculateBonus() << " FCFA" << endl;
    }
};

class PartTimeEmployee : public Employee {
private:
    int hoursWorked;

public:
    PartTimeEmployee(string n, int i, double s, int hours)
        : Employee(n, i, s), hoursWorked(hours) {}

    double calculateBonus() const override {
        return hoursWorked * 100;
    }

    void displayDetails() const override {
        Employee::displayDetails();
        cout << "Hours Worked: " << hoursWorked << endl;
        cout << "Bonus: " << calculateBonus() << " FCFA" << endl;
    }
};
```

(c) Composition: Department Class

```
class Department {
private:
    string departmentName;
    vector<Employee*> employees;

public:
    Department(string name) : departmentName(name) {}

    void addEmployee(Employee* emp) {
        employees.push_back(emp);
    }

    void displayDepartmentDetails() const {
        cout << "Department: " << departmentName << endl;
        cout << "Employees:" << endl;
        for (Employee* emp : employees) {
            emp->displayDetails();
            cout << "---" << endl;
        }
    }

    ~Department() {
        for (Employee* emp : employees) {
            delete emp;
        }
    }
};
```

Demonstration in main():

```
int main() {
    Department dept("Engineering");

    Employee* emp1 = new FullTimeEmployee("Alice", 101, 50000,
5);
    Employee* emp2 = new PartTimeEmployee("Bob", 102, 30000, 40);

    dept.addEmployee(emp1);
    dept.addEmployee(emp2);

    dept.displayDepartmentDetails();
}
```

```
        return 0;
    }
```

Output:

```
Department: Engineering
Employees:
Name: Alice
ID: 101
Salary: 50000 FCFA
Years of Service: 5
Bonus: 25000 FCFA
---
Name: Bob
ID: 102
Salary: 30000 FCFA
Hours Worked: 40
Bonus: 4000 FCFA
---
```

Q7: Animal Kingdom Simulation

(a) Base Class: Animal

```
class Animal {
protected:
    string name;
    int age;

public:
    Animal(string n, int a) : name(n), age(a) {}

    virtual void makeSound() const = 0;
    virtual void move() const = 0;

    virtual void displayDetails() const {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << " years" << endl;
    }

    virtual ~Animal() {}
};
```

(b) Derived Classes: Mammal, Bird, and Fish

```
class Mammal : public Animal {
public:
    Mammal(string n, int a) : Animal(n, a) {}

    void makeSound() const override {
        cout << name << " makes a mammal sound!" << endl;
    }

    void move() const override {
        cout << name << " runs on land." << endl;
    }
};

class Bird : public Animal {
public:
    Bird(string n, int a) : Animal(n, a) {}

    void makeSound() const override {
        cout << name << " chirps!" << endl;
    }

    void move() const override {
        cout << name << " flies in the air." << endl;
    }
};

class Fish : public Animal {
public:
    Fish(string n, int a) : Animal(n, a) {}

    void makeSound() const override {
        cout << name << " makes no sound (silent)." << endl;
    }

    void move() const override {
        cout << name << " swims in water." << endl;
    }
};
```

(c) Polymorphism in Action

```
int main() {
    Animal* animal1 = new Mammal("Lion", 5);
    Animal* animal2 = new Bird("Eagle", 3);
    Animal* animal3 = new Fish("Salmon", 2);

    vector<Animal*> animals = {animal1, animal2, animal3};

    for (Animal* animal : animals) {
        animal->displayDetails();
        animal->makeSound();
        animal->move();
        cout << "---" << endl;
    }

    for (Animal* animal : animals) {
        delete animal;
    }

    return 0;
}
```

Output:

```
Name: Lion
Age: 5 years
Lion makes a mammal sound!
Lion runs on land.
---
Name: Eagle
Age: 3 years
Eagle chirps!
Eagle flies in the air.
---
Name: Salmon
Age: 2 years
Salmon makes no sound (silent).
Salmon swims in water.
---
```

Summary of OOP Concepts

Concept	Example in Code	Purpose
Encapsulation	Private/protected attributes in all classes	Protects data integrity (e.g., balance in Account).
Inheritance	FictionBook inherits from Book	Reuse code and extend functionality (e.g., adding genre to Book).
Polymorphism	displayDetails() in Book and derived classes	Same method name, different implementations (e.g., SavingsAccount vs CurrentAccount).
Abstraction	Pure virtual functions in Shape and Animal	Forces derived classes to implement their own logic (e.g., area()).
Composition	Department contains Employee objects	Models "has-a" relationships (e.g., a department has employees).
Dynamic Binding	Virtual functions in all base classes	Enables runtime polymorphism (e.g., calling makeSound() for any Animal).

How to Use This Canvas

1. **Edit Directly:** Modify the content as needed (e.g., add/remove questions, update code).
2. **Export to PDF:**
 - Use **Typora** or **Pandoc** to convert this Markdown to PDF.
 - Copy to **Microsoft Word** and save as PDF.
3. **Share or Print:** Use the rendered output for study or reference.